

進階方法

- **21-1:使用同名方法**
 - 同名方法(Overload).....21-1
 - 定義同名方法.....21-2
 - 使用同名方法.....21-3
- **21-2:Generic泛用方法**
 - 泛用(Generic).....21-4
 - 泛用方法.....21-5
 - 宣告泛用方法.....21-6
 - 使用泛用方法.....21-7
- **21-3:選擇性參數**
 - 參數預設值.....21-8
 - 使用選擇性參數.....21-9

Module 21

同名方法(Overload)

- 或稱方法多載，具有相同名稱，但參數個數或型態不同的方法。
- 優點：
 - 方便使用。
 - 可以輕鬆地為現有的類別方法加入新功能。
- 缺點：
 - 不易除錯。
 - 不易維護。

定義同名方法

- 在相同名稱的方法定義不同的參數個數或型態。

```
static string MyOverload()  
{  
    return "哈哈是我啦!";  
}  
static string MyOverload(string s1)  
{  
    return s1;  
}  
static string MyOverload(string s1, string s2)  
{  
    return s1 + s2;  
}
```

使用同名方法

- 輸入不同的參數則會呼叫不同的方法。

```
static void Main(string[] args)
{
    Console.WriteLine(MyOverload());
    Console.WriteLine(MyOverload("哈囉"));
    Console.WriteLine(MyOverload("是在","哈囉"));

    Console.ReadKey();
}
```



泛用(Generic)

- 泛型 (Generic) 是從 .NET Framework 2.0 開始引入的型別參數 (Type Parameter) 機制。
- 當在設計類別或方法時，若無法預先知道使用者會傳入哪種資料型態，就可使用泛型來延後型別決定時機，提高程式的彈性與重用性。
- 避免重複撰寫相同邏輯的多型別版本程式。
- 提高程式可讀性與可維護性。
- 泛型可以套用到class, struct, interface, delegate, 方法。

泛用方法

- 應用在方法上的泛型，稱為泛用方法(**Generic Method**)。
- 泛用方法不用事先指定參數的資料型態，可在呼叫時由呼叫端動態決定型態。
- 同一個泛用方法可被不同型態的參數重複使用，提高程式彈性與重用性。

```
static void MyGenericVoid<T>(T para)
{
    Console.WriteLine(para.ToString());
}
```

```
static void Main(string[] args)
{
    MyGenericVoid<int>( 666);
    MyGenericVoid<string>("泛用型別");
    Console.ReadKey();
}
```

宣告泛用方法

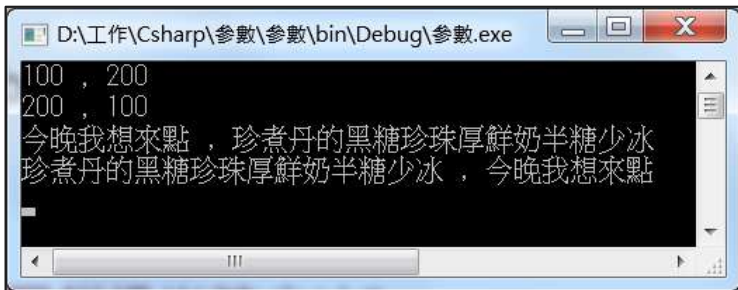
- < >內放泛用型態，就像是型態變數。
- < >內可放多個泛用型態。

```
static void Swap<T>(ref T a, ref T b)
{
    T temp = a;
    a = b;
    b = temp;
}
```

```
static void Swap<T, Y>(ref T a, ref Y b)
{
    Console.WriteLine(a + ", " + b);
}
```

使用泛用方法

- 與一般呼叫方法相同。



```
static void Main(string[] args)
{
    int a = 100, b = 200;
    Console.WriteLine(a + " , " + b);
    Swap<int>(ref a, ref b);
    Console.WriteLine(a + " , " + b);

    string s1 = "今晚我想來點";
    string s2 = "珍煮丹的黑糖珍珠厚鮮奶半糖少冰";
    Console.WriteLine(s1 + " , " + s2);
    Swap(ref s1, ref s2);
    Console.WriteLine(s1 + " , " + s2);
    Console.ReadKey();
}
```


參數預設值

- 方法參數可設定預設值，若使用者未輸入參數，則方法將帶入預設參數。

```
static int MyDefault(int Top = 10, int Bot = 20, int Height = 5)
{
    return (Top + Bot) * Height / 2;
}
```

```
static void Main(string[] args)
{
    int area = MyDefault();
    Console.WriteLine("梯形面積" + area);

    Console.ReadKey();
}
```



使用選擇性參數

- 沒有預設值的參數呼叫時必須輸入，反之有預設值的參數可選擇性輸入，稱為選擇性參數。

```
static void Main(string[] args)
{
    int area = MyDefault(10, 25);
    Console.WriteLine("梯形面積" + area);

    Console.ReadKey();
}

static int MyDefault(int Top, int Bot = 20, int Height = 5)
{
    return (Top + Bot) * Height / 2;
}
```